48

分布式系统

分布式系统改变了世界的面貌。你的 Web 浏览器连接到地球另一端的某台 Web 服务器时，它参与的就是一种看似简单的**客户端/服务器**（client/server）分布式系统。不过，当你访问 Google 或 Facebook 这样的现代 Web 服务时，与你交互的并不只是一台机器；在幕后，这些复杂的服务由一大批（成千上万台）机器构成，它们相互协作，共同提供这个网站的服务。由此不难看出，研究分布式系统为什么这么有意思。事实上，这个主题值得单独开一门课；这里我们只介绍其中几个主要话题。

构建分布式系统会遇到许多新的挑战。我们关注的主要挑战是**故障**（failure）：机器、磁盘、网络和软件都会时不时出故障，因为我们不知道（而且很可能永远都不会知道）怎样构建“完美”的组件和系统。然而，构建现代 Web 服务时，我们希望在客户端看来，它从不出故障。怎样才能做到这一点呢?

关键问题：
如何构建在组件出故障时仍能工作的系统

怎样用并非一直正常工作的部件，构建出一个能正常工作的系统? 这个基本问题应该会让你想起我们在 RAID 存储阵列中讨论过的一些话题；不过，这里的问题往往更复杂，解决方案也是如此。

有意思的是，故障虽然是构建分布式系统的核心挑战，但它也代表着一种机遇。没错，机器会出故障；但一台机器出故障，并不意味着整个系统都必须跟着出故障。把一组机器组织起来，我们就能构建出一个看起来很少出故障的系统，尽管它的组件经常出故障。这正是分布式系统的核心之美和价值所在，也是几乎每一个你在用的现代 Web 服务（包括 Google、Facebook 等）都建立在分布式系统之上的原因。

提示：通信本质上是不可靠的

几乎在任何情况下，都应该把通信看作一种从根本上不可靠的活动。比特损坏、链路和机器宕机或无法工作、缺少缓冲区空间来容纳传入的数据包，这些都会导致同一个结果：数据包有时到不了目的地。要在这样不可靠的网络之上构建可靠的服务，就必须考虑能够应对丢包的技术。

此外还有其他重要问题。系统**性能**（performance）往往至关重要；分布式系统靠网络连接在一起，系统设计者常常必须仔细考虑如何完成给定的任务，尽量减少发送的消息数，并让通信尽可能高效（低延迟、高带宽）。

最后，**安全**（security）也是必须考虑的问题。连接到远程站点时，能否确认对方确实是它所声称的那个身份，就成了核心问题。此外，确保第三方无法监视或篡改另外两方之间正在进行的通信，也是一项挑战。

在这篇介绍中，我们要讨论分布式系统中最基本的新问题：**通信**（communication）。也就是说，分布式系统中的机器应该如何彼此通信？我们会从最基本的原语，即消息讲起，并在其上构建几个更高层的原语。如上所述，故障将是讨论的重点：通信层应该如何处理故障？

48.1 通信基础

现代网络的核心信条是：通信从根本上就是不可靠的。无论是在广域的互联网上，还是在 Infiniband 这样的局域高速网络中，数据包都经常会丢失、损坏，或者因为其他原因到不了目的地。

导致数据包丢失或损坏的原因多种多样。有时，传输过程中由于电气或其他类似问题，一些比特发生了翻转。有时，系统中的某个部件，比如网络链路、路由器，甚至远程主机，出于某种原因损坏了或者无法正常工作；网线确实会被意外切断，至少偶尔会发生。

然而，更根本的丢包原因，是网络交换机、路由器或端点内部的缓冲不足。具体来说，即使我们能保证所有链路都正常工作，系统中所有的组件（交换机、路由器、终端主机）都按预期正常运行，仍然可能丢包，原因如下。假设一个数据包到达一台路由器；为了处理这个数据包，必须把它放到路由器内部的某块内存里。如果同时有很多这样的数据包到达，路由器里的内存就可能容纳不下所有数据包。这时，路由器唯一的选择就是**丢弃**（drop）其中一个或多个数据包。终端主机上也会发生同样的情况：你向一台机器发送大量消息时，这台机器的资源很容易就会不堪重负，于是又会丢包。

因此，丢包是网络中的基本问题。于是问题变成了：我们应该如何应对丢包？

48.2 不可靠的通信层

一个简单的办法是：不去应对它。有些应用程序知道如何处理丢包，所以有时让它们通过一个基本的、不可靠的消息层来通信也很有用，这就是人们常说的**端到端原则**（end-to-end argument）的一个例子（见本章末尾的**补充**）。这种不可靠通信层的一个绝佳例子，就是如今几乎所有现代系统上都有的 **UDP/IP** 网络协议栈。要使用 UDP，进程要通过**套接字**（sockets）API 创建一个**通信端点**（communication endpoint）；其他机器上（或者同一台机器上）的进程向这个进程发送 UDP **数据报**（datagram）。数据报就是一条大小有上限的定长消息。

图 48.1 和图 48.2 展示了一个构建在 UDP/IP 之上的简单客户端和服务器。客户端可以向服务器发送一条消息，服务器随后返回一条回复。只用这么一点代码，你就拥有了开始构建分布式系统所需的一切！

 © 2008–23, ARPACI-DUSSEAU

```
// 客户端代码
int main(int argc, char *argv[]) {
  int sd = UDP_Open(20000);
  struct sockaddr_in addrSnd, addrRcv;
  int rc = UDP_FillSockAddr(&addrSnd, "cs.wisc.edu", 10000);
  char message[BUFFER_SIZE];
  sprintf(message, "hello world");
  rc = UDP_Write(sd, &addrSnd, message, BUFFER_SIZE);
  if (rc > 0)
    int rc = UDP_Read(sd, &addrRcv, message, BUFFER_SIZE);
  return 0;
}

// 服务器代码
int main(int argc, char *argv[]) {
  int sd = UDP_Open(10000);
  assert(sd > -1);
  while (1) {
    struct sockaddr_in addr;
    char message[BUFFER_SIZE];
    int rc = UDP_Read(sd, &addr, message, BUFFER_SIZE);
    if (rc > 0) {
      char reply[BUFFER_SIZE];
      sprintf(reply, "goodbye world");
      rc = UDP_Write(sd, &addr, reply, BUFFER_SIZE);
    }
  }
  return 0;
}
```

图 48.1：**UDP 示例代码（client.c、server.c）**

UDP 是不可靠通信层的一个很好的例子。使用 UDP 时，你会遇到数据包丢失（被丢弃）、因而到不了目的地的情况，而发送方永远不会得知丢包的消息。不过，这并不意味着 UDP 对任何故障都毫无防范。例如，UDP 带有一个**校验和**（checksum），用来检测某些形式的数据包损坏。

然而，许多应用程序只是想把数据发送到目的地，而不想操心丢包问题，所以我们还需要更多机制。具体来说，我们需要在不可靠的网络之上实现可靠的通信。

48.3 可靠的通信层

要构建可靠的通信层，我们需要一些新的机制和技术来处理丢包。我们来看一个简单的例子：客户端通过一条不可靠的连接向服务器发送一条消息。首先要回答的问题是：发送方怎么知道接收方确实收到了消息？

```
int UDP_Open(int port) {
  int sd;
  if ((sd = socket(AF_INET, SOCK_DGRAM, 0)) == -1)
    return -1;
  struct sockaddr_in myaddr;
  bzero(&myaddr, sizeof(myaddr));
  myaddr.sin_family      = AF_INET;
  myaddr.sin_port        = htons(port);
  myaddr.sin_addr.s_addr = INADDR_ANY;
  if (bind(sd, (struct sockaddr *) &myaddr,
           sizeof(myaddr)) == -1) {
    close(sd);
    return -1;
  }
  return sd;
}

int UDP_FillSockAddr(struct sockaddr_in *addr,
                     char *hostname, int port) {
  bzero(addr, sizeof(struct sockaddr_in));
  addr->sin_family = AF_INET;          // 主机字节序
  addr->sin_port   = htons(port);      // 网络字节序
  struct in_addr *in_addr;
  struct hostent *host_entry;
  if ((host_entry = gethostbyname(hostname)) == NULL)
    return -1;
  in_addr = (struct in_addr *) host_entry->h_addr;
  addr->sin_addr = *in_addr;
  return 0;
}

int UDP_Write(int sd, struct sockaddr_in *addr,
              char *buffer, int n) {
  int addr_len = sizeof(struct sockaddr_in);
  return sendto(sd, buffer, n, 0, (struct sockaddr *)
                addr, addr_len);
}

int UDP_Read(int sd, struct sockaddr_in *addr,
             char *buffer, int n) {
  int len = sizeof(struct sockaddr_in);
  return recvfrom(sd, buffer, n, 0, (struct sockaddr *)
                  addr, (socklen_t *) &len);
}
```

图 48.2: **一个简单的 UDP 库（`udp.c`）**

提示：用校验和保证完整性

在现代系统中，校验和是一种常用的方法，可以快速、有效地检测数据损坏。最简单的校验和就是加法：把一块数据的所有字节加起来；当然，人们还设计出了许多更复杂的校验和，包括基本的循环冗余校验（CRC）、Fletcher 校验和等等 [MK09]。

在网络中，校验和的用法如下。从一台机器向另一台机器发送消息之前，先对消息的字节计算一个校验和，然后把消息和校验和一起发往目的地。在目的地，接收方同样对收到的消息计算校验和；如果算出的校验和与发来的校验和一致，接收方就可以比较放心，认为数据在传输过程中很可能没有损坏。

评价校验和可以从几个不同的角度入手。有效性是一个首要考虑：数据发生变化时，校验和会不会随之改变？校验和越强，数据的变化就越难逃过检测。性能是另一个重要标准：计算校验和的代价有多大？遗憾的是，有效性和性能往往相互矛盾，也就是说，高质量的校验和计算起来往往代价很高。生活，又一次不完美。

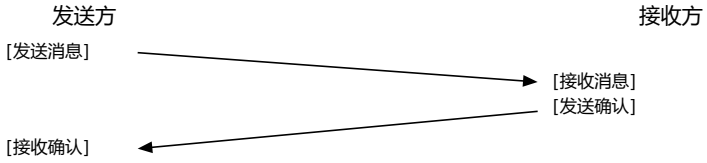


图 48.3：**消息加确认**

我们要用的技术叫作**确认**（acknowledgment），简称 **ack**。思路很简单：发送方向接收方发送一条消息；接收方随后回发一条简短的消息，*确认*自己收到了。图 48.3 描绘了这个过程。

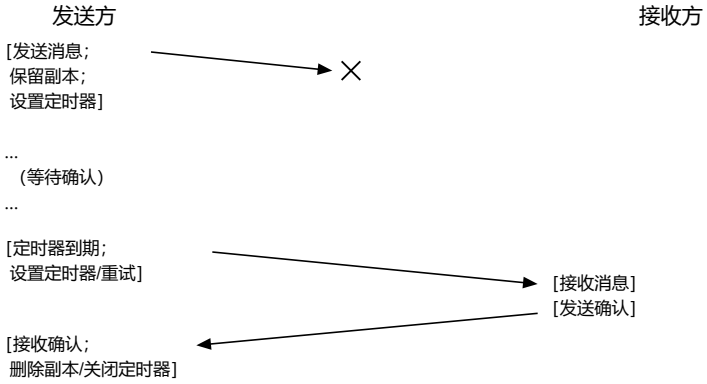


图 48.4：**消息加确认：请求丢失**

 © 2008–23, ARPACI-DUSSEAU

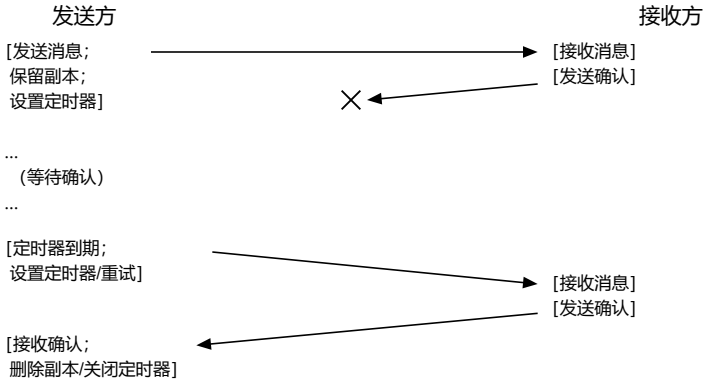


图 48.5：**消息加确认：回复丢失**

发送方收到对这条消息的确认后，就可以放心，接收方确实收到了原来的消息。可是，如果发送方没有收到确认，它该怎么办?

为了处理这种情况，我们还需要一个机制，叫作**超时**（timeout）。发送方发送消息时，会设置一个定时器，让它在一段时间后到期。如果在这段时间内没有收到确认，发送方就断定消息已经丢失，然后简单地**重试**（retry）发送，把同一条消息再发一遍，希望这次能送达。为了让这种方法奏效，发送方必须保留一份消息副本，以备需要再次发送。超时和重试结合在一起，有人就把这种方法叫作**超时/重试**（timeout/retry）；搞网络的那帮人可真聪明，不是吗？图 48.4 给出了一个例子。

遗憾的是，这种形式的超时/重试还不够。图 48.5 给出了一个可能引发麻烦的丢包例子。在这个例子中，丢失的不是原来的消息，而是确认。从发送方的角度看，情况似乎是一样的：没有收到确认，因此应该超时并重试。但从接收方的角度看，情况就大不相同了：同一条消息被收到了两次！虽然有些情况下这样也没关系，但一般来说是不行的；想象一下，你下载一个文件时，下载内容里重复出现了多余的数据包，会发生什么。因此，当我们的目标是一个可靠的消息层时，通常还希望保证接收方对每条消息**恰好接收一次**（exactly once）。

为了让接收方能够检测出重复传输的消息，发送方必须以某种唯一的方式标识每条消息，接收方也需要某种办法来记录自己是否已经见过每条消息。接收方发现重复传输时，只是确认这条消息，但（关键在于）不把它交给接收数据的应用程序。这样，发送方收到了确认，而消息又不会被接收两次，从而保持了上面说的恰好一次语义。

检测重复消息的办法有很多。例如，发送方可以为每条消息生成一个唯一的 ID；接收方可以记录它见过的每一个 ID。这种方法可行，但代价高得令人望而却步，因为要记录所有 ID，需要无限多的内存。

有一种更简单、几乎不占内存的方法可以解决这个问题，这种机制叫作 **sequence counter**。使用 sequence counter 时，发送方和接收方先约定一个计数器的起始值（例如 1），双方各自维护这个计数器。每发送一条消息，就把计数器的当前值随消息一起发出去；

 © 2008–23, ARPACI-DUSSEAU

提示：设置超时值要小心

从上面的讨论中你大概也能猜到，正确设置超时值，是用超时来重发消息时的一个重要方面。超时值设得太小，发送方就会做无谓的重发，浪费发送方的 CPU 时间和网络资源。超时值设得太大，发送方等待重发的时间就太长，发送方感受到的性能也会因此下降。因此，对单个客户端和服务器来说，“合适”的超时值，就是等到刚好足以发现丢包为止，不再多等。

然而，正如我们在后面的章节中会看到的，分布式系统中往往不止一个客户端和一个服务器。在许多客户端向同一台服务器发送消息的场景中，服务器端的丢包可能意味着服务器已经过载。如果是这样，客户端也许应该换一种自适应的方式重试；例如，第一次超时之后，客户端可以把超时值调大一些，也许调到原来的两倍。这种**指数退避**（exponential back-off）方案最早在早期的 Aloha 网络中使用，后来被早期的以太网采用 [A70]，它可以避免过多的重发让资源过载。健壮的系统都会尽力避免这类过载。

这个计数器值（N）就充当这条消息的 ID。消息发出之后，发送方把计数器的值加一（变成 $N+1$）。

接收方把自己的计数器值当作来自该发送方的下一条消息的预期 ID。如果收到的消息的 ID（N）与接收方的计数器值（同样是 N）相等，接收方就确认这条消息，并把它向上交给应用程序；在这种情况下，接收方断定这是第一次收到这条消息。然后，接收方把计数器加一（变成 $N+1$），等待下一条消息。

如果确认丢失了，发送方会超时并重发消息 N。这一次，接收方的计数器值更大（$N+1$），于是接收方知道自己已经收到过这条消息。所以，它确认这条消息，但不把它向上交给应用程序。就这样，用 sequence counter 这种简单的方式就能避免重复。

最常用的可靠通信层叫作 **TCP/IP**，简称 **TCP**。TCP 比上面描述的要复杂得多，包括处理网络拥塞的机制 [VJ88]、多个未完成的请求，以及数以百计的其他小调整和优化。如果你好奇，可以去多读一些相关资料；更好的办法是去上一门网络课程，把这些内容学扎实。

48.4 通信抽象

有了基本的消息层，我们接下来讨论本章的下一个问题：构建分布式系统时，应该使用什么样的通信抽象？

多年来，系统研究界提出了许多方法。其中一类工作把操作系统的抽象拿过来，扩展到分布式环境中运行。例如，**分布式共享内存**（distributed shared memory，**DSM**）系统让不同机器上的进程能够共享一个很大的虚拟地址空间 [LH89]。这种抽象把分布式计算变成了看起来像多线程应用程序的东西；唯一的区别是，这些线程运行在不同的机器上，而不是同一台机器的不同处理器上。

大多数 DSM 系统都是通过操作系统的虚拟内存系统来工作的。在一台机器上访问某一页时，可能出现两种情况。第一种（最好的）情况是，这一页已经在本机上，于是数据很快就能取到。第二种情况是，这一页目前在其他某台机器上。这时会发生缺页异常，缺页异常

处理程序向其他某台机器发送消息，取回这一页，把它装入发出请求的进程的页表，然后继续执行。

由于种种原因，这种方法如今并没有得到广泛使用。DSM 最大的问题在于它如何处理故障。例如，设想一台机器出了故障，这台机器上的页会怎么样？如果分布式计算的数据结构分散在整个地址空间中呢？在这种情况下，这些数据结构的一部分会突然变得不可用。地址空间的一部分消失了，要应对这样的故障是很难的；想象一个链表，其中某个“next”指针指向了地址空间中已经不存在的那部分。天哪！

另一个问题是性能。人们写代码时，通常假设访问内存的代价很低。在 DSM 系统中，有些访问代价很低，但另一些会引发缺页异常，需要代价高昂地从远程机器取数据。因此，这类 DSM 系统的程序员必须非常小心地组织计算，让几乎不发生任何通信，而这在很大程度上违背了采用这种方法的初衷。虽然这个领域做了大量研究，但实际影响甚微；如今没有人用 DSM 构建可靠的分布式系统。

48.5 远程过程调用（RPC）

事实证明，操作系统的抽象并不适合用来构建分布式系统，而编程语言（PL）的抽象则要合理得多。最主流的抽象建立在**远程过程调用**（remote procedure call，简称 **RPC**）这一思想之上 [BN84][1]。

所有远程过程调用软件包都有一个简单的目标：让在远程机器上执行代码的过程，像调用本地函数一样简单直接。因此，在客户端看来，它发起一次过程调用，过一段时间后，结果就返回了。服务器只需定义一些它想导出的例程，剩下的魔法都由 RPC 系统处理。RPC 系统一般由两部分组成：**存根生成器**（stub generator，有时也叫 **protocol compiler**）和**运行时库**（run-time library）。下面我们分别详细地看看这两部分。

存根生成器

存根生成器的任务很简单：把函数参数和结果打包成消息这件事自动化，省掉其中的一些麻烦。这样做好处很多：从设计上就避免了手工编写这类代码时容易犯的简单错误；此外，存根编译器或许还能优化这些代码，从而提高性能。

这种编译器的输入，就是服务器想导出给客户端的一组调用。从概念上讲，它可以简单到像这样：

```
interface {
  int func1(int arg1);
  int func2(int arg1, int arg2);
};
```

存根生成器拿到这样的接口后，会生成几段不同的代码。对客户端来说，会生成一个**客户端存根**（client stub），其中包含接口里列出的每一个函数；想使用这个 RPC 服务的客户端程序，只要链接这个客户端存根，并调用其中的函数，就能发起 RPC。

[1] 在现代编程语言中，我们也许会改称**远程方法调用**（remote method invocation，**RMI**），可是话说回来，谁会喜欢这些满是花哨对象的语言呢？

在内部，客户端存根中的每个函数都会完成执行远程过程调用所需的全部工作。在客户端看来，代码只是一次函数调用（例如，客户端调用 `func1(x)`）；在内部，客户端存根中 `func1()` 的代码会做这些事：

- **创建消息缓冲区**。消息缓冲区通常就是一个有一定大小的连续字节数组。
- **把所需的信息打包进消息缓冲区**。这些信息包括要调用的函数的某种标识符，以及这个函数所需的全部参数（例如，在上面的例子中，`func1` 需要一个整数）。把这些信息全部放进一个连续缓冲区的过程，有时称为参数的**编组**（marshaling）或消息的**序列化**（serialization）。
- **把消息发送给目标 RPC 服务器**。与 RPC 服务器的通信，以及让通信正确运行所需的全部细节，都由 RPC 运行时库处理，下面会进一步介绍。
- **等待回复**。因为函数调用通常是**同步的**（synchronous），所以调用会等待它完成。
- **解包返回码和其他参数**。如果函数只返回一个返回码，这个过程就很直接；但更复杂的函数可能返回更复杂的结果（例如一个列表），因此存根可能还需要把这些结果解包。这一步也叫**解组**（unmarshaling）或**反序列化**（deserialization）。
- **返回到调用者**。最后，从客户端存根返回到客户端代码中。

服务器端也会生成代码。服务器上的步骤如下：

- **解包消息**。这一步叫作**解组**或**反序列化**，从传入的消息中取出信息，提取出函数标识符和参数。
- **调用实际的函数**。终于到了！现在终于真正执行远程函数了。RPC 运行时根据 ID 调用指定的函数，并传入所需的参数。
- **打包结果**。把返回的参数编组回一个回复缓冲区。
- **发送回复**。最后把回复发给调用者。

存根编译器还要考虑另外几个重要问题。第一个是复杂参数，也就是说，如何打包并发送一个复杂的数据结构？例如，调用 `write()` 系统调用时，要传入三个参数：一个整数文件描述符、一个指向缓冲区的指针，以及一个表示要写入多少字节（从指针处开始）的大小。如果传给 RPC 软件包的是一个指针，它就得能弄清楚如何解释这个指针，并执行正确的操作。通常的做法有两种：一种是使用众所周知的类型（例如 RPC 编译器能够理解的 `buffer_t`，用来按给定大小传递数据块）；另一种是给数据结构加上更多注解信息，让编译器知道哪些字节需要序列化。

另一个重要问题是服务器在并发方面如何组织。简单的服务器只是在一个简单的循环里等待请求，一次处理一个请求。但你可能已经猜到，这样做效率可能极低：如果一次 RPC 调用阻塞了（例如在 I/O 上阻塞），服务器资源就被浪费了。因此，大多数服务器都会以某种并发的方式构建。一种常见的组织方式是**线程池**（thread pool）。在这种组织方式下，服务器启动时创建一组数量有限的线程；消息到达时，把它分派给其中一个工作线程，由这个工作线程完成 RPC 调用的工作，最后返回回复；在此期间，主线程继续接收其他请求，并可能把它们分派给其他工作线程。这样的组织方式让服务器内部可以并发执行，从而提高了服务器的利用率；常见的代价也随之而来，主要是编程复杂度，因为 RPC 调用现在可能需要使用锁和其他同步原语来确保正确运行。

运行时库

运行时库承担了 RPC 系统中的大部分繁重工作；大多数性能和可靠性问题都在这里处理。下面讨论构建这样一个运行时层的几个主要挑战。

我们首先要克服的挑战之一，是如何找到远程服务。这个**命名**（naming）问题在分布式系统中很常见，在某种意义上也超出了我们当前讨论的范围。最简单的方法建立在现有的命名系统之上，例如当前互联网协议提供的主机名和端口号。在这样的系统中，客户端必须知道运行所需 RPC 服务的机器的主机名或 IP 地址，以及该服务使用的端口号（端口号只是标识一台机器上某一项特定通信活动的方式，让一台机器能同时拥有多个通信通道）。然后，协议栈必须提供一种机制，把数据包从系统中的任何其他机器路由到某个特定地址。关于命名的深入讨论，你得去别处找，例如去了解互联网上的 DNS 和名字解析，更好的办法是直接读 Saltzer 和 Kaashoek 那本书中精彩的一章 [SK09]。

客户端知道了某项远程服务该找哪台服务器之后，下一个问题是：RPC 应该建立在哪种传输层协议之上？具体来说，RPC 系统应该使用 TCP/IP 这样的可靠协议，还是建立在 UDP/IP 这样的不可靠通信层之上？

乍一看，这个选择似乎很简单：我们显然希望请求能可靠地送达远程服务器，也显然希望能可靠地收到回复。所以，我们应该选择 TCP 这样的可靠传输协议，对吧？

遗憾的是，在可靠通信层之上构建 RPC，可能导致性能上严重的低效。回想一下上面讨论的可靠通信层是如何工作的：靠确认加上超时/重试。因此，客户端向服务器发送 RPC 请求时，服务器会回复一个确认，好让调用者知道请求已经收到。同样，服务器向客户端发送回复时，客户端也会确认，好让服务器知道回复已经收到。在可靠通信层之上构建请求/响应协议（比如 RPC），会多发送两条“额外”的消息。

因此，许多 RPC 软件包都建立在 UDP 这样的不可靠通信层之上。这样做可以实现更高效的 RPC 层，但也要由 RPC 系统来承担提供可靠性的责任。RPC 层通过超时/重试和确认来达到所需的可靠性，做法和上面描述的差不多。通过某种形式的序号，通信层可以保证每次 RPC 恰好执行一次（在没有故障的情况下），或者至多执行一次（在发生故障的情况下）。

其他问题

RPC 运行时还必须处理其他一些问题。例如，如果一次远程调用需要很长时间才能完成，会怎么样？考虑到我们的超时机制，一个长时间运行的远程调用在客户端看来可能就像发生了故障，从而触发重试，所以这里需要小心处理。一种解决办法是，当回复无法立即生成时，使用一个显式的确认（由接收方发给发送方），让客户端知道服务器已经收到了请求。然后，过一段时间之后，客户端可以定期询问服务器是否还在处理这个请求；如果服务器一直回答“是”，客户端就应该放心地继续等待（毕竟，过程调用有时确实要执行很久才能完成）。

运行时还必须处理参数很大的过程调用，这些参数大到单个数据包装不下。有些较低层的网络协议提供了这种发送方的**分片**（fragmentation，把较大的数据包分成一组较小的数据包）和接收方的**重组**（reassembly，把较小的部分重新组合成一个较大的逻辑整体）；如果没有，RPC 运行时可能就得自己实现这种功能。详情请参阅 Birrell 和 Nelson 的论文 [BN84]。

 © 2008–23, ARPACI-DUSSEAU

补充：端到端原则

端到端原则认为，在分层系统中，最高层（通常就是位于“端”上的应用程序）最终是唯一能够真正实现某些功能的地方。在他们那篇具有里程碑意义的论文 [SRC84] 中，Saltzer 等人用一个绝佳的例子来论证这一点：两台机器之间的可靠文件传输。如果你想把一个文件从机器 A 传到机器 B，并且确保最终落在 B 上的字节与从 A 出发时完全相同，你就必须对此做一次“端到端”的检查；更底层的可靠机制，例如网络或磁盘中的可靠机制，并不能提供这样的保证。

与之相对的方法，是试图通过给系统的较低层增加可靠性来解决可靠文件传输问题。例如，假设我们构建了一个可靠的通信协议，并用它来实现可靠的文件传输。这个通信协议保证发送方发出的每一个字节都会被接收方按顺序收到，比如通过超时/重试、确认和序号来实现。遗憾的是，用了这样的协议，并不能造就可靠的文件传输：想象一下，在通信发生之前，字节就已经在发送方的内存中损坏了；或者接收方把数据写到磁盘时出了什么岔子。在这些情况下，即使字节在网络上被可靠地传送了，我们的文件传输最终也是不可靠的。要构建可靠的文件传输，就必须加入端到端的可靠性检查，例如，在整个传输完成之后，把接收方磁盘上的文件读回来，计算校验和，再和发送方文件的校验和比较。

这条准则的推论是，有时让较低层提供额外的功能，确实可以提高系统性能，或者以其他方式优化系统。因此，你不应该排除在系统的较低层设置这类机制；而应该结合它在整个系统或应用中的最终用途，仔细考虑它的效用。

许多系统都要处理的一个问题是**字节序**（byte ordering）。你可能知道，有些机器按所谓的**大端**（big endian）顺序存储数值，另一些则使用**小端**（little endian）顺序。大端从最高有效位到最低有效位依次存储（比如一个整数的）各个字节，很像阿拉伯数字的写法；小端则正好相反。两种方式都是存储数值信息的有效方式；这里的问题是，字节序不同的机器之间如何通信。

RPC 软件包通常在消息格式中规定一种明确的字节序来解决这个问题。在 Sun 的 RPC 软件包中，**XDR**（**eXternal Data Representation**，外部数据表示）层提供了这一功能。如果发送或接收消息的机器的字节序与 XDR 一致，消息就照常发送和接收。但如果通信的机器字节序不同，消息中的每一项信息都必须转换。因此，字节序的差异会带来少许性能开销。

最后一个问题是，要不要把通信的异步本质暴露给客户端，以便进行一些性能优化。具体来说，典型的 RPC 是**同步**（synchronously）进行的，也就是说，客户端发起过程调用后，必须等过程调用返回才能继续执行。因为这种等待可能很长，而客户端也许还有其他工作可做，所以有些 RPC 软件包允许你**异步**（asynchronously）地发起 RPC。发起异步 RPC 时，RPC 软件包发出请求后立即返回；客户端随后可以自由地做其他工作，比如调用其他 RPC 或进行其他有用的计算。客户端到了某个时候会想看看异步 RPC 的结果；于是它再调用 RPC 层，让它等待未完成的 RPC 完成，然后就可以访问返回的参数了。

 © 2008–23, ARPACI-DUSSEAU

48.6 小结

我们介绍了一个新的主题：分布式系统，以及它的主要问题：如何处理已经变得司空见惯的故障。就像 Google 内部的说法：如果你只有一台台式机，故障很少见；而当你身处一个有成千上万台机器的数据中心时，故障无时无刻不在发生。任何分布式系统的关键，都在于你如何应对故障。

我们还看到，通信是一切分布式系统的核心。远程过程调用（RPC）是对这种通信的一种常见抽象，它让客户端能够在服务器上发起远程调用；RPC 软件包处理所有繁琐的细节，包括超时/重试和确认，以提供一种与本地过程调用十分相像的服务。

要真正理解 RPC 软件包，最好的办法当然是亲自用一用。Sun 的 RPC 系统使用存根编译器 `rpcgen`，算是比较老的一个；Google 的 gRPC 和 Apache Thrift 则是同类系统的现代实现。挑一个试试，看看它们到底有什么了不起。

 © 2008–23, ARPACI-DUSSEAU

参考文献

[A70] "The ALOHA System — Another Alternative for Computer Communications" by Norman Abramson. The 1970 Fall Joint Computer Conference. *The ALOHA network pioneered some basic concepts in networking, including exponential back-off and retransmit, which formed the basis for communication in shared-bus Ethernet networks for years.*

[BN84] "Implementing Remote Procedure Calls" by Andrew D. Birrell, Bruce Jay Nelson. ACM TOCS, Volume 2:1, February 1984. *The foundational RPC system upon which all others build. Yes, another pioneering effort from our friends at Xerox PARC.*

[MK09] "The Effectiveness of Checksums for Embedded Control Networks" by Theresa C. Maxino and Philip J. Koopman. IEEE Transactions on Dependable and Secure Computing, 6:1, January '09. *A nice overview of basic checksum machinery and some performance and robustness comparisons between them.*

[LH89] "Memory Coherence in Shared Virtual Memory Systems" by Kai Li and Paul Hudak. ACM TOCS, 7:4, November 1989. *The introduction of software-based shared memory via virtual memory. An intriguing idea for sure, but not a lasting or good one in the end.*

[SK09] "Principles of Computer System Design" by Jerome H. Saltzer and M. Frans Kaashoek. Morgan-Kaufmann, 2009. *An excellent book on systems, and a must for every bookshelf. One of the few terrific discussions on naming we've seen.*

[SRC84] "End-To-End Arguments in System Design" by Jerome H. Saltzer, David P. Reed, David D. Clark. ACM TOCS, 2:4, November 1984. *A beautiful discussion of layering, abstraction, and where functionality must ultimately reside in computer systems.*

[VJ88] "Congestion Avoidance and Control" by Van Jacobson. SIGCOMM '88. *A pioneering paper on how clients should adjust to perceived network congestion; definitely one of the key pieces of technology underlying the Internet, and a must read for anyone serious about systems, and for Van Jacobson's relatives because well relatives should read all of your papers.*

 © 2008–23, ARPACI-DUSSEAU

作业（编程）

在这一部分，我们会写一些简单的通信代码，让你熟悉怎么做这件事。玩得开心！

问题

1. 用本章提供的代码，构建一个简单的基于 UDP 的服务器和客户端。服务器应该接收来自客户端的消息，并回复一个确认。第一次尝试时，不要加任何重传或健壮性机制（假设通信完全正常）。先在一台机器上运行测试，之后再在两台不同的机器上运行。
2. 把你的代码改造成一个**通信库**（communication library）。具体来说，设计你自己的 API，包括发送和接收调用，以及其他需要的 API 调用。改写你的客户端和服务器，让它们使用你的库，而不是直接调用原始的套接字接口。
3. 以**超时/重试**的形式，给你日渐成形的通信库加上可靠通信。具体来说，你的库应该为每条要发送的消息保留一份副本。发送时，应该启动一个定时器，以便记录消息发出后过了多长时间。在接收方，库应该**确认**收到的消息。客户端在发送时应该**阻塞**，也就是说，它应该等到消息被确认之后才返回。它还应该愿意无限次地重试发送。消息的最大长度应该是用 UDP 能发送的最大单条消息的长度。最后，一定要高效地实现超时/重试：让调用者睡眠，直到确认到达或传输超时；不要自旋浪费 CPU！
4. 让你的库更高效、功能更丰富。首先，加入超大消息的传输。具体来说，尽管网络限制了消息的最大长度，你的库也应该能接收任意大小的消息，并把它从客户端传到服务器。客户端应该把这些大消息分成若干片段发给服务器；服务器端的库代码应该把收到的片段组装成连续的整体，再把这个大缓冲区交给等待中的服务器代码。
5. 再做一遍上面的事，但要做到高性能。不要一次只发一个片段，而应该快速地发送许多片段，从而让网络得到充分得多的利用。为此，要仔细标记传输中的每一个片段，确保接收方重组时不会把消息弄乱。
6. 最后一个实现上的挑战：按序交付的异步消息发送。也就是说，客户端应该能够反复调用发送函数，一条接一条地发送消息；接收方应该调用接收函数，可靠地按顺序收到每一条消息；发送方的许多消息应该可以同时在网络上传输。再加一个发送方的调用，让客户端可以等待所有未完成的消息都被确认。
7. 现在还有一个痛点：测量。测量你每种方法的带宽：在两台不同的机器之间能传输多少数据，速率是多少？再测量延迟：发送单个数据包并收到确认，要多久才能完成？最后，你的数据看起来合理吗？你原本的预期是什么？怎样才能更好地设定预期，以便判断是有问题，还是你的代码运行良好？

 © 2008–23, ARPACI-DUSSEAU